**INSTITUTO SUPERIOR TECNOLÓGICO DE LA JOYA**

TALLER DE LENGUAJES DE PROGRAMACIÓN

# INFORME DE LA SESIÓN 2

## PROYECTO: GAMESTORE ESCOLAR – CATÁLOGO Y TIENDA DE RECOMPENSAS

**Alumno:** Marcos Riquelme Huahuiza Sarayasi

**Archivo:** CatalogoTiendaAvanzado.java

## 1. Introducción

En esta sesión armé la ventana del catálogo de recompensas de GameStore Escolar usando Java Swing. En la guía anterior ubicaba cada botón con coordenadas, y esta vez aprendí a usar Layouts, que acomodan los componentes solos y hacen que la ventana se vea ordenada aunque cambie de tamaño.

## 2. Objetivo

Hacer una interfaz ordenada en Java Swing usando JPanel y JScrollPane como contenedores, BorderLayout y GridLayout para acomodar todo, y los componentes JComboBox, JCheckBox y JRadioButton para que el usuario pueda elegir opciones.

## 3. Herramientas que usé

- **Lenguaje:** Java, con el JDK versión 17 o superior.
- **Librería gráfica:** Swing (javax.swing y java.awt).
- **Editor:** Visual Studio Code con la extensión Extension Pack for Java.
- **Ejecución:** la terminal de VS Code, con los comandos javac y java.

## 4. Conceptos que se usaron

### 4.1 Administradores de diseño

Un Layout Manager es el que decide dónde va cada componente y qué tamaño tiene. Yo solo le doy la regla y Swing hace el cálculo, incluso si la ventana se agranda o se achica. Usé tres: BorderLayout, que divide la ventana en cinco zonas (NORTH, SOUTH, EAST, WEST y CENTER); GridLayout, que arma una cuadrícula con celdas del mismo tamaño; y FlowLayout, que pone los componentes uno al lado del otro y los baja a otra línea cuando ya no caben.

### 4.2 Componentes del programa

| Componente | Función en el programa |
|---|---|
| JFrame | Es la ventana principal "GameStore Escolar - Catálogo Avanzado", de 550 x 450. |
| BorderLayout | Divide la ventana en zonas: filtros arriba, catálogo al centro y botón de canje abajo. |
| JPanel + GridLayout(3, 2, 10, 10) | Acomoda los 6 botones de recompensas en 3 filas y 2 columnas. |
| JScrollPane | Pone barras de desplazamiento cuando el catálogo no cabe en la ventana. |
| JComboBox | Lista desplegable para filtrar por categoría: Todas, Avatars, Pases y Accesorios. |
| JCheckBox | Casilla "Recibir alerta por correo", que se puede marcar o no. |
| JRadioButton + ButtonGroup | Opciones "Digital" y "Físico" agrupadas, así que solo se puede marcar una. |

## 5. Desarrollo

### 5.1 Preparación

Primero creé la carpeta Sesion-02 y dentro de ella otra llamada codigo. Después abrí esa carpeta en Visual Studio Code con Archivo → Abrir carpeta y creé el archivo CatalogoTiendaAvanzado.java. El nombre del archivo tiene que ser igual al de la clase, si no Java da error al compilar.

### 5.2 El código

La clase CatalogoTiendaAvanzado hereda de JFrame. En el constructor configuro la ventana con BorderLayout. En la parte de arriba (NORTH) está el panel de filtros con el JComboBox, el JCheckBox y los dos JRadioButton juntos en un ButtonGroup. En el centro (CENTER) están los 6 botones de recompensas dentro de un GridLayout, y todo eso metido en un JScrollPane. Abajo (SOUTH) está el botón "Canjear Ítem".

```
import javax.swing.*;
import java.awt.*;

public class CatalogoTiendaAvanzado extends JFrame {
    private JComboBox<String> cmbCategorias;
    private JCheckBox chkNotificaciones;
    private JRadioButton rbtDigital, rbtFisico;
    private ButtonGroup grupoEnvio;
    private JPanel panelFiltros, panelCentro, panelSur;

    public CatalogoTiendaAvanzado() {
        setTitle("GameStore Escolar - Catálogo Avanzado");
        setSize(550, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        panelFiltros = new JPanel(new FlowLayout());
        cmbCategorias = new JComboBox<>(new String[]{"Todas las categorías",
"Avatars", "Pases", "Accesorios"});
        panelFiltros.add(new JLabel("Filtrar:"));
        panelFiltros.add(cmbCategorias);

        chkNotificaciones = new JCheckBox("Recibir alerta por correo");
        panelFiltros.add(chkNotificaciones);

        rbtDigital = new JRadioButton("Digital", true);
        rbtFisico = new JRadioButton("Físico");
        grupoEnvio = new ButtonGroup();
        grupoEnvio.add(rbtDigital);
        grupoEnvio.add(rbtFisico);
        panelFiltros.add(rbtDigital);
        panelFiltros.add(rbtFisico);
        add(panelFiltros, BorderLayout.NORTH);

        panelCentro = new JPanel(new GridLayout(3, 2, 10, 10));
```

```
        panelCentro.add(new JButton("Skin de Avatar (100 pts)"));
        panelCentro.add(new JButton("Pase de Tarea (200 pts)"));
        panelCentro.add(new JButton("Cursor Personalizado (50 pts)"));
        panelCentro.add(new JButton("Insignia Dorada (150 pts)"));
        panelCentro.add(new JButton("Fondo Exclusivo (80 pts)"));
        panelCentro.add(new JButton("Bonus Sorpresa (300 pts)"));

        JScrollPane scrollPane = new JScrollPane(panelCentro);
        add(scrollPane, BorderLayout.CENTER);

        panelSur = new JPanel();
        panelSur.add(new JButton("Canjear Ítem"));
        add(panelSur, BorderLayout.SOUTH);
    }

    public static void main(String[] args) {
        new CatalogoTiendaAvanzado().setVisible(true);
    }
}
```

*Figura 1. Código de CatalogoTiendaAvanzado.java en VS Code.*

### 5.3 Compilar y ejecutar

Con la terminal de VS Code ya ubicada en la carpeta codigo, escribí estos dos comandos:

```
javac CatalogoTiendaAvanzado.java
java CatalogoTiendaAvanzado
```

El primero compila el archivo y crea CatalogoTiendaAvanzado.class. El segundo ejecuta el programa y abre la ventana.

### 5.4 Resultado

Al ejecutarlo se abre la ventana con los filtros arriba, los 6 productos en una cuadrícula de 3 filas por 2 columnas y el botón "Canjear Ítem" abajo. Si se reduce la altura de la ventana, aparece la barra de desplazamiento del JScrollPane. Los controles se pueden marcar y elegir, pero todavía no hacen nada, porque no les agregué ActionListener.

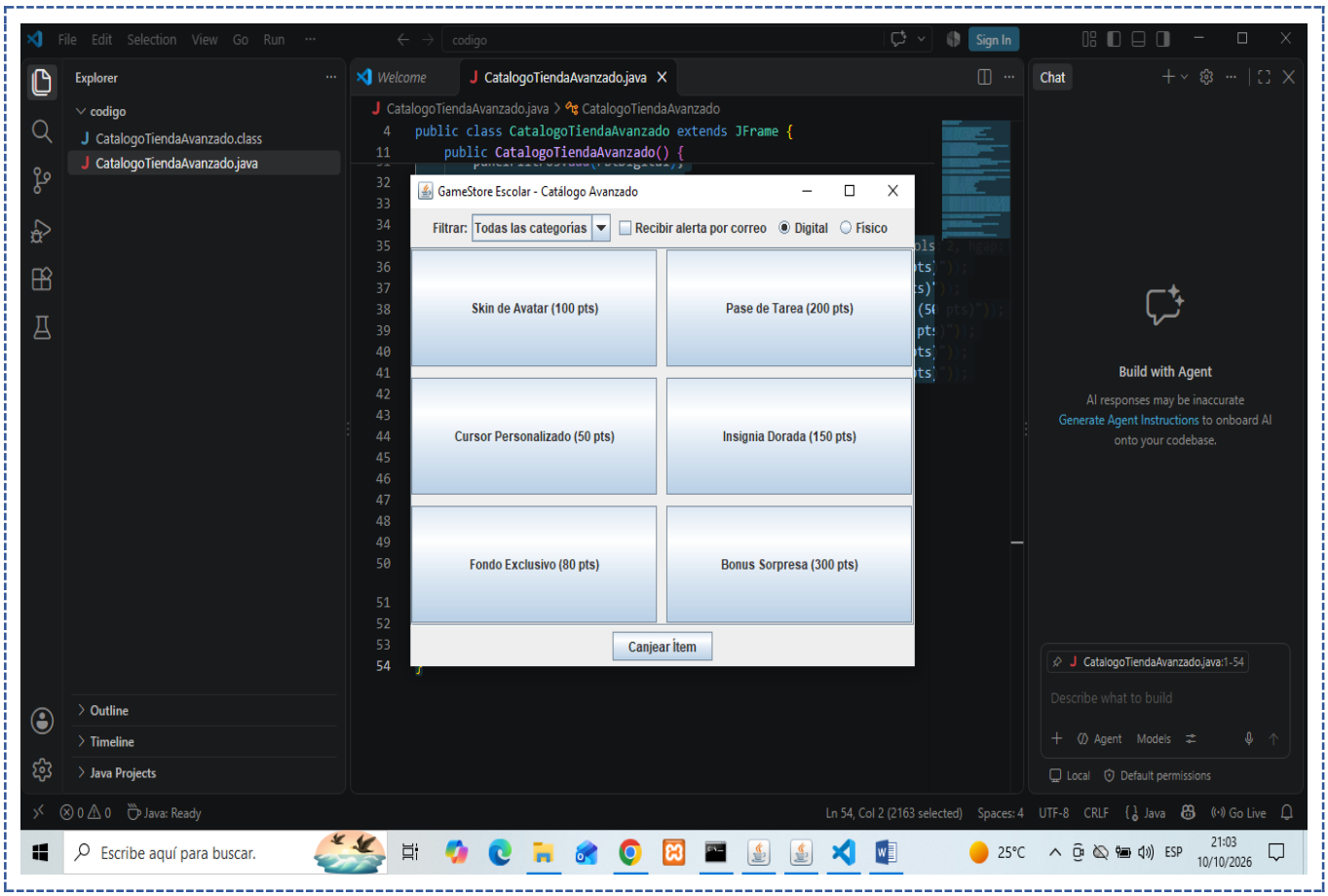



*Figura 2. Ventana del catálogo ejecutándose.*

## 6. ¿Por qué usar Layouts y no coordenadas fijas?

En la Guía 1 usé setLayout(null) con setBounds(), que pone cada componente en un punto exacto de la pantalla. El problema es que si la ventana cambia de tamaño, los componentes se quedan donde estaban y todo se desordena. También puede verse mal en otra pantalla con distinta resolución, y si quiero agregar un botón nuevo tengo que volver a calcular la posición de los demás.

Con los Layouts esto ya no pasa, porque yo defino la regla y Swing hace el trabajo. En este proyecto, BorderLayout mantiene los filtros arriba, el catálogo al centro y el botón abajo. GridLayout reparte los productos en celdas iguales. FlowLayout baja los controles del filtro a otra línea cuando la ventana se angosta, y el JScrollPane permite ver todo el contenido aunque no entre completo.

Por eso creo que los Layouts son mejor opción: el código queda más ordenado, es más fácil de modificar y se pueden agregar productos sin rehacer todo el diseño.

## 7. Conclusiones

- Pude armar la ventana combinando BorderLayout, GridLayout, JPanel y JScrollPane.
- Agregué un JComboBox, un JCheckBox y dos JRadioButton agrupados con ButtonGroup, y se ven y se pueden usar en la ventana.
- Los Layouts hacen que la interfaz se adapte mejor y que sea más fácil de mantener que usando coordenadas fijas.
- Por ahora los controles solo se ven; en una siguiente sesión les agregaré su función con ActionListener.